

Department of Computer Engineering

ISOLA Game in Prolog with Alpha-Beta Algorithm

51620 : Radesh Govind (A51620@alunos.isel.pt)

51619 : Martim Monteiro (A51619@alunos.isel.pt)

Report for the Course Unit of Artificial Intelligence
of the Bachelor's Degree in Computer Science and Engineering

Professor : Professor Nuno Leite

Abstract

This report documents the implementation of the ISOLA strategy board game in SWI-Prolog, developed as part of the Artificial Intelligence course. The game state is represented as a list of lists in Prolog, with predicates for move generation, move application, and end-game detection. The computer player's artificial intelligence is based on the alpha-beta algorithm, an efficient implementation of the minimax principle, enhanced with heuristic move ordering. The evaluation function combines mobility and board centrality. A unit test suite comprising 33 tests was executed, with all tests passing. Experimental benchmarks across varying board sizes and search depths demonstrate that depth 4 is tractable on 3×3 boards (136 ms), but becomes prohibitive on 6×6 boards (approximately 167 s for the first move), making depth 2 the recommended compromise for larger boards.

Contents

List of Figures	ix
List of Tables	xi
List of Listings	xiii
1 Introduction	1
1.1 The ISOLA Game	1
1.1.1 Game Rules	1
1.1.2 Game Example	1
1.2 Learning Objectives	2
1.3 Introduction to Prolog	2
1.3.1 Analogy: The Detective and the Clues	2
1.3.2 Terms and Unification	3
1.3.3 Backtracking	3
1.3.4 Lists in Prolog	4
1.4 Report Structure	4
2 Implementation	5
2.1 Board Representation (<code>board.pl</code>)	5
2.1.1 Data Structure	5
2.1.2 Main Predicates	6
2.2 Game Rules (<code>rules.pl</code>)	6
2.2.1 Move Validation	6
2.2.2 Move Application and Square Removal	7
2.2.3 End-Game Condition	7
2.3 Artificial Intelligence (<code>ai.pl</code>)	8
2.3.1 The Minimax Principle	8

2.3.1.1	Concept and Analogy	8
2.3.1.2	Operation	8
2.3.1.3	Terminal Values	9
2.3.2	The Alpha-Beta Algorithm	9
2.3.2.1	Concept and Analogy	9
2.3.2.2	The Parameters α and β	9
2.3.2.3	Implementation	10
2.3.2.4	Performance Gain	11
2.3.3	Move Ordering	11
2.3.4	Evaluation Function	11
2.4	Game Interface (<code>game.pl</code>)	12
2.4.1	Program Flow	12
2.4.2	Human Turn vs. AI Turn	13
2.5	Use of Artificial Intelligence in Development	13
3	Experimental Evaluation	15
3.1	Unit Tests	15
3.1.1	Execution and Results	15
3.1.2	Test Coverage	16
3.1.3	Unit Test Example	16
3.2	Performance Benchmarks	17
3.2.1	Methodology	17
3.2.2	Results	18
3.2.3	Results Analysis	18
3.2.3.1	Actual Branching Factor	18
3.2.3.2	Impact of Depth	18
3.2.3.3	Consistency of Selected Moves	18
3.2.3.4	Heuristic Function Evaluation	19
3.3	Limitations and Future Work	19
3.3.1	Identified Limitations	19
3.3.2	Configuration Recommendations	19
3.4	Conclusions	20
	References	21

A Source Code	i
A.1 board.pl — Board	i
A.2 rules.pl — Game Rules	i
A.3 ai.pl — Alpha-Beta Algorithm	ii

List of Figures

1.1	Progression of an ISOLA 6×6 game. The symbol # represents a permanently removed square.	2
2.1	Valid moves from different positions (X = valid destination, P = pawn).	7
2.2	Example of a minimax tree with 2 levels. The MAX player selects the branch with the maximum value; the MIN player selects the branch with the minimum value. The value at the root is 3.	8
2.3	Alpha cutoff: the MAX player has already guaranteed the value 3 (left branch). In the right branch, the MIN node found 2, which is worse for MAX. The remaining children of the right branch are discarded.	10

List of Tables

2.1	Project module structure	5
2.2	Complexity: minimax vs. alpha-beta	11
3.1	Distribution and coverage of unit tests	17
3.2	Computation time for the first AI move	18
3.3	Evaluation function values at test positions	19

List of Listings

1.1	Basic facts and rules in Prolog	2
1.2	Unification examples	3
1.3	Backtracking demonstration	3
1.4	findall collects all solutions	4
1.5	3×3 board as a list of lists	4
2.1	Initial board creation	6
2.2	Non-destructive cell update	6
2.3	Move validation	6
2.4	Apply a move and remove a square	7
2.5	End-game detection	7
2.6	Pure minimax implementation	8
2.7	Terminal values	9
2.8	Alpha-beta search — recursive case	10
2.9	Scoring function for move ordering	11
2.10	Heuristic evaluation function	12
2.11	Main game loop	13
3.1	Test: moves from the centre of a 3×3 board	16
A.1	board.pl — board creation and manipulation	i
A.2	rules.pl — move validation and application	i
A.3	ai.pl — entry point and alpha-beta search	ii

Chapter 1

Introduction

1.1 The ISOLA Game

ISOLA (also known as *Isolation* or *Solitude*) is a two-player strategy game created by Bernd Kienitz in 1972 [2]. The objective is straightforward: isolate the opponent's pawn such that it cannot make any valid move on its turn.

1.1.1 Game Rules

The board is a square grid (typically 6×6). Each player has a pawn that starts at opposite corners:

- **Player 1** (x) starts at position (row 0, column 0) — top-left corner.
- **Player 2** (o) starts at position (row 5, column 5) — bottom-right corner.

On each turn, a player must perform exactly **two actions**:

1. **Move** their pawn to an adjacent free square (horizontally, vertically, or diagonally — like a king in chess, but only one step at a time).
2. **Remove** any empty square from the board (except the squares occupied by the pawns), marking it as permanently inaccessible (#).

It is not permitted to jump over the opponent or to enter already-removed squares. The player who leaves the opponent with no valid move at the start of their turn wins.

1.1.2 Game Example

The example below illustrates the progression of a game on a 6×6 grid. In the initial state, Player 1 moves from (0,0) to (1,1) and removes square (2,2).

The strategy consists of keeping one's own pawn with many movement options (**mobility**) while progressively reducing the opponent's mobility, ultimately isolating them completely.

Initial state							After P1 moves (0,0)→(1,1), removes (2,2)						
	0	1	2	3	4	5		0	1	2	3	4	5
	+-----							+-----					
0		x	.	.	.	.	0		.	.	.	.	.
1		.	.	.	.	.	1		.	x	.	.	.
2		.	.	.	.	.	2		.	.	#	.	.
3		.	.	.	.	.	3		.	.	.	.	.
4		.	.	.	.	.	4		.	.	.	.	.
5		.	.	.	.	o	5		.	.	.	.	o

Figure 1.1: Progression of an ISOLA 6×6 game. The symbol # represents a permanently removed square.

1.2 Learning Objectives

This work aims to consolidate the following learning objectives:

1. Understand and master the syntax of Prolog programs.
2. Use basic constructs: atoms, compound terms, lists, operators, and arithmetic.
3. Understand and control *backtracking*.
4. Be able to use typical built-in predicates (`findall/3`, `nth0/3`, `maplist/3`, etc.).
5. Implement a two-player game with perfect information.
6. Understand the *minimax* principle.
7. Understand the *alpha-beta* algorithm: an efficient implementation of *minimax*.

1.3 Introduction to Prolog

To understand the implementation, it is essential to understand the fundamentals of Prolog [1, 5]. Unlike languages such as Java or Python, Prolog does not describe how to compute a result, but rather what is true. The runtime system is responsible for determining how to derive the answer.

1.3.1 Analogy: The Detective and the Clues

Consider Prolog as a detective. One provides it with a set of **facts** (known clues) and **rules** (logical deductions), and then poses **queries** (questions). The detective systematically searches all possible combinations until an answer is found.

```

1 % Facts: who likes what
2 likes(mary, cats).
3 likes(john, dogs).
4 likes(mary, dogs).
5
6 % Rule: two people have something in common

```

```

7 in_common(X, Y, Thing) :-
8     likes(X, Thing),
9     likes(Y, Thing),
10    X \= Y.
11
12 % Query: ?- in_common(mary, john, Thing).
13 % Answer: Thing = dogs.

```

Listing 1.1: Basic facts and rules in Prolog

The clause beginning with `:-` is read as “if”: `in_common(X, Y, Thing)` is **true if** `likes(X, Thing)` is true **and** `likes(Y, Thing)` is true **and** `X` is not equal to `Y`.

1.3.2 Terms and Unification

The central element of Prolog is the **term**. There are four types:

- **Atom** — symbolic constant: `cat`, `x`, `'.'`, `true`.
- **Number** — integer or floating-point: `3`, `4.5`.
- **Variable** — begins with an uppercase letter or underscore: `X`, `Player`, `_`.
- **Compound term** — functor with arguments: `move(1/2, 3/4)`, `[a,b,c]`.

Unification is the mechanism by which Prolog binds values to variables. When one writes `X = 3`, Prolog unifies `X` with `3`. Unification succeeds only if the two terms share the same structure.

```

1 ?- X = 5.           % X = 5 (succeeds)
2 ?- 2/3 = R/C.       % R = 2, C = 3 (term decomposition)
3 ?- [H|T] = [1,2,3]. % H = 1, T = [2,3] (head and tail)
4 ?- 2 = 3.           % false (unification fails)

```

Listing 1.2: Unification examples

1.3.3 Backtracking

Backtracking is Prolog’s search mechanism. When an attempt to satisfy a goal fails, Prolog **backtracks** automatically and tries the next available alternative.

Analogy: imagine a maze. Prolog tries one path; if it reaches a dead end, it backtracks and tries the next corridor, until all paths have been explored.

```

1 color(red).
2 color(green).
3 color(blue).
4
5 % Query: ?- color(X).
6 % X = red (first solution)

```

```

7 % X = green   (after requesting more: ";")
8 % X = blue    (third solution)
9 % false       (no more solutions)

```

Listing 1.3: Backtracking demonstration

The predicate `findall/3` exploits *backtracking* to collect all solutions into a list, which is used extensively throughout this project:

```

1 ?- findall(X, color(X), List).
2 % List = [red, green, blue]

```

Listing 1.4: `findall` collects all solutions

1.3.4 Lists in Prolog

In Prolog, a list is represented as `[Elem1, Elem2, ...]`. The notation `[H|T]` separates the head (H) from the tail (T).

In the ISOLA game, the board is a **list of lists**: each row of the board is a list of atoms.

```

1 Board = [
2   [x, '.', '.'], % row 0
3   ['.', '.', '.'], % row 1
4   ['.', '.', o] % row 2
5 ]
6 % Symbols: '.' = free, x = P1 pawn,
7 %          o = P2 pawn, '#' = removed

```

Listing 1.5: 3×3 board as a list of lists

1.4 Report Structure

This report is organised as follows:

- **Chapter 2 — Implementation:** Describes in detail all project modules: board representation, game rules, artificial intelligence algorithms, and the game interface.
- **Chapter 3 — Experimental Evaluation:** Presents the results of the unit tests and performance benchmarks obtained under different configurations.
- **Appendix A — Source Code:** Selected listings of the developed Prolog files.

Chapter 2

Implementation

This chapter describes the implementation of the ISOLA game in SWI-Prolog. The project is organised into five modules:

Table 2.1: Project module structure

File	Responsibility	Main predicates
board.pl	Board representation and display	initial_board, get_cell, set_cell
rules.pl	Game rules and move validation	valid_move, apply_move, game_over
ai.pl	Artificial intelligence (alpha-beta)	best_move, alphabeta, evaluate
game.pl	Interface and game loop	play, game_loop, human_turn
main.pl	Entry point	initialization

2.1 Board Representation (board.pl)

2.1.1 Data Structure

The board is represented as a **list of lists** in Prolog. Each element of the outer list is a row of the board; each element of a row is an atom indicating the contents of the square:

- ' .' — free (empty) square
- x — Player 1's pawn
- o — Player 2's pawn
- '#' — permanently removed square

Positions are represented as the compound term `Row/Column` (zero-based indexing). For example, `2/3` denotes row 2, column 3.

Advantage of this representation: lists are Prolog's native data structure, and predicates such as `nth0/3` (index-based access) and `maplist/2` (functional application) are available in the standard library, greatly simplifying the code.

2.1.2 Main Predicates

The predicate `initial_board/2` creates a board of size $N \times N$ entirely filled with free squares:

```

1 initial_board(Size, Board) :-
2     length(Board, Size),           % Board has Size rows
3     maplist(make_row(Size), Board). % each row has Size columns
4
5 make_row(Size, Row) :-
6     length(Row, Size),
7     maplist(=('.'), Row).          % each cell is '.'

```

Listing 2.1: Initial board creation

How it works: `length(Board, Size)` creates a list of `Size` uninstantiated variables. `maplist` applies `make_row` to each element, and within each row, `maplist(=('.'), Row)` unifies each cell with the atom `'.'`.

Cell updates are **non-destructive**: Prolog does not modify existing structures, but constructs a new version of the board. This is a fundamental characteristic of logic programming.

```

1 set_cell(Board, Row, Col, Value, NewBoard) :-
2     nth0(Row, Board, OldRow, RestRows), % extract row Row
3     nth0(Col, OldRow, _, RestCols),      % extract element Col
4     nth0(Col, NewRow, Value, RestCols),  % rebuild row with new value
5     nth0(Row, NewBoard, NewRow, RestRows).% rebuild board

```

Listing 2.2: Non-destructive cell update

Analogy: consider a film frame. When a character moves, the previous frame is not erased — a new frame is created with the character in the new position. Prolog operates in the same way.

2.2 Game Rules (`rules.pl`)

2.2.1 Move Validation

A move is valid if the destination square:

1. is exactly one step away (in any of the 8 directions);
2. is within the board boundaries;
3. is free (`'.'`).

```

1 valid_move(Board, FromR, FromC, ToR, ToC) :-
2     board_size(Board, Size),
3     Max is Size - 1,
4     member(DR, [-1, 0, 1]), % row delta: -1, 0 or +1
5     member(DC, [-1, 0, 1]), % column delta: -1, 0 or +1

```



```

6  \+ (DR == 0, DC == 0),    % cannot stay in place
7  ToR is FromR + DR,
8  ToC is FromC + DC,
9  ToR >= 0, ToR <= Max,      % within boundaries
10 ToC >= 0, ToC <= Max,
11 get_cell(Board, ToR, ToC, '.'). % free square

```

Listing 2.3: Move validation

This predicate functions both as a **verifier** (when ToR/ToC are already instantiated) and as a **generator** (when they are unbound, Prolog generates all combinations of DR/DC through *backtracking*). This duality is one of the most elegant features of Prolog.

Pawn at centre: 8 moves	Pawn at corner: 3 moves
X X X	P X .
X P X	X X .
X X X	. . .

Figure 2.1: Valid moves from different positions (X = valid destination, P = pawn).

2.2.2 Move Application and Square Removal

```

1 % Move the pawn from From to To
2 apply_move(Board, FromR, FromC, ToR, ToC, NewBoard) :-
3     get_cell(Board, FromR, FromC, Piece), % read the symbol
4     set_cell(Board, FromR, FromC, '.', B1), % clear origin
5     set_cell(B1, ToR, ToC, Piece, NewBoard). % place at destination
6
7 % Mark a square as permanently removed
8 apply_remove(Board, Row, Col, NewBoard) :-
9     set_cell(Board, Row, Col, '#', NewBoard).

```

Listing 2.4: Apply a move and remove a square

2.2.3 End-Game Condition

The game ends when the player whose turn it is has no valid move:

```

1 game_over(Board, Player, P1Pos, P2Pos) :-
2     player_pos(Player, P1Pos, P2Pos, CR/CC),
3     \+ valid_move(Board, CR, CC, _, _).
4     % \+ means negation: true if valid_move FAILS

```

Listing 2.5: End-game detection

The operator `\+` is Prolog's **negation as failure**: "`\+ Goal`" is true if and only if `Goal` fails. Here, `game_over` is true when no valid move exists.

2.3 Artificial Intelligence (ai.pl)

2.3.1 The Minimax Principle

2.3.1.1 Concept and Analogy

Minimax is the classical algorithm for two-player games with perfect information [4].

Analogy: consider playing chess and deciding the best move. One reasons: “If I play here, my opponent will respond with their best move, then I will respond with mine, and so on.” The *minimax* algorithm formalises exactly this reasoning.

2.3.1.2 Operation

The algorithm mentally constructs a **game tree**:

- **MAX nodes** (Player 1, symbol x): chooses the move that **maximises** the score.
- **MIN nodes** (Player 2, symbol o): chooses the move that **minimises** the score (disadvantaging Player 1).
- **Leaf nodes** (depth 0 or game over): the position is evaluated by a heuristic function.

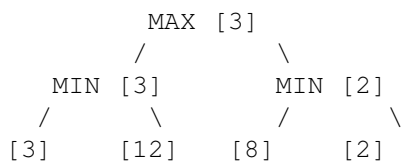


Figure 2.2: Example of a minimax tree with 2 levels. The MAX player selects the branch with the maximum value; the MIN player selects the branch with the minimum value. The value at the root is 3.

```

1 % Base case 1: depth exhausted -> static evaluation
2 minimax(Board, _Player, P1Pos, P2Pos, 0, Value) :-
3     !, evaluate(Board, P1Pos, P2Pos, Value).
4
5 % Base case 2: player has no moves -> terminal loss
6 minimax(Board, Player, P1Pos, P2Pos, _D, Value) :-
7     player_pos(Player, P1Pos, P2Pos, CR/CC),
8     \+ valid_move(Board, CR, CC, _, _), !,
9     terminal_value(Player, Value).
10
11 % Recursive case: generate children and apply max/min
12 minimax(Board, Player, P1Pos, P2Pos, Depth, Value) :-
13     all_move_pairs(Board, Player, P1Pos, P2Pos, Pairs),
14     NextD is Depth - 1,
15     Other is 3 - Player,      % 1->2, 2->1

```

```

16  maplist(minimax_child(Board, Player, Other,
17              P1Pos, P2Pos, NextD),
18              Pairs, ChildValues),
19  ( Player == 1
20  -> max_list(ChildValues, Value) % MAX selects the highest
21  ; min_list(ChildValues, Value) % MIN selects the lowest
22  ) .

```

Listing 2.6: Pure minimax implementation

2.3.1.3 Terminal Values

When a player runs out of moves, the game terminates:

```

1 terminal_value(1, -1000). % Player 1 trapped -> loss (negative)
2 terminal_value(2, 1000). % Player 2 trapped -> P1 wins (positive)

```

Listing 2.7: Terminal values

The values ± 1000 are intentionally much larger than any heuristic evaluation value, ensuring that an actual win or loss is always preferred over a state with a favourable heuristic.

2.3.2 The Alpha-Beta Algorithm

2.3.2.1 Concept and Analogy

Minimax is correct but slow: it explores every node in the tree. *Alpha-beta* is an optimisation that discards branches that **can never influence the final result**, without altering the computed value.

Analogy: imagine selecting the best restaurant from a list. One has already visited a restaurant rated 8/10. Upon visiting the second restaurant, a dish rated 3/10 is encountered. Without examining the rest of the menu, one can conclude that this restaurant will never be the best option. **The remainder of the visit can be skipped.** Alpha-beta applies exactly this reasoning to branches of the game tree.

2.3.2.2 The Parameters α and β

- α (*alpha*) — the best value the MAX player has already guaranteed. Initialised to $-\infty$.
- β (*beta*) — the best value the MIN player has already guaranteed. Initialised to $+\infty$.
- **Beta cutoff** (at a MAX node): if the current value $\geq \beta$, the MIN player would never choose this branch. Search can stop.
- **Alpha cutoff** (at a MIN node): if the current value $\leq \alpha$, the MAX player would never choose this branch. Search can stop.

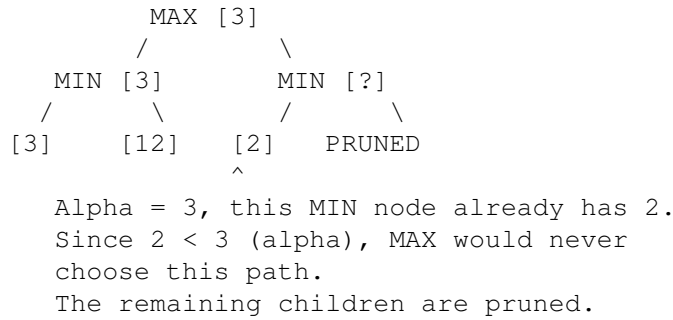


Figure 2.3: Alpha cutoff: the MAX player has already guaranteed the value 3 (left branch). In the right branch, the MIN node found 2, which is worse for MAX. The remaining children of the right branch are discarded.

2.3.2.3 Implementation

```

1 alphabeta(Board, Player, P1Pos, P2Pos, Depth, Alpha, Beta,
2     Value, BM, BR) :-
3     all_move_pairs(Board, Player, P1Pos, P2Pos, Pairs),
4     order_pairs(Pairs, Board, Player, P1Pos, P2Pos, Ordered),
5     NextDepth is Depth - 1,
6     Other is 3 - Player,
7     ( Player == 1
8     -> ab_max(Ordered, Board, Other, P1Pos, P2Pos,
9         Player, NextDepth, Alpha, Beta,
10        -10000, none, none, Value, BM, BR)
11    ;   ab_min(Ordered, Board, Other, P1Pos, P2Pos,
12        Player, NextDepth, Alpha, Beta,
13        10000, none, none, Value, BM, BR)
14    ).
15
16 % MAX node: update alpha and cut if alpha >= beta
17 ab_max([move(MPos,RPos)|Rest], Board, Other, P1Pos, P2Pos,
18     CurPlayer, Depth, Alpha, Beta, BestVal, BestM, BestR,
19     Val, BM, BR) :-
20     apply_pair(Board, CurPlayer, P1Pos, P2Pos, MPos, RPos,
21         NewBoard, NewP1, NewP2),
22     alphabeta(NewBoard, Other, NewP1, NewP2, Depth,
23         Alpha, Beta, ChildVal, _, _),
24     ( (BestM = none ; ChildVal > BestVal)
25     -> NewBest = ChildVal, NewBM = MPos, NewBR = RPos
26     ;   NewBest = BestVal, NewBM = BestM, NewBR = BestR
27     ),
28     NewAlpha is max(Alpha, NewBest),
29     ( NewAlpha >= Beta
30     -> Val = NewBest, BM = NewBM, BR = NewBR % beta cutoff
31     ;   ab_max(Rest, Board, Other, P1Pos, P2Pos,
32         CurPlayer, Depth, NewAlpha, Beta,

```

```

33         NewBest, NewBM, NewBR, Val, BM, BR)
34     ) .

```

Listing 2.8: Alpha-beta search — recursive case

2.3.2.4 Performance Gain

Alpha-beta pruning drastically reduces the number of evaluated nodes [3]:

Table 2.2: Complexity: minimax vs. alpha-beta

Algorithm	Complexity	Nodes (b=8, d=4)
Pure minimax	$O(b^d)$	$8^4 = 4,096$
Ideal alpha-beta	$O(b^{d/2})$	$8^2 = 64$

In practice, given that each move has on average 8 possible destinations and 34 removable squares on an initial 6×6 board, the actual branching factor is ≈ 102 (see Chapter 3). Alpha-beta with good move ordering reduces this factor to $\approx \sqrt{102} \approx 10$ per level.

2.3.3 Move Ordering

The effectiveness of alpha-beta pruning depends strongly on the order in which moves are evaluated. Evaluating the most promising moves first maximises the number of cutoffs.

Implemented heuristic:

- **Central moves first:** squares near the centre of the board offer more future options (greater mobility).
- **Removals close to the opponent first:** removing squares adjacent to the opponent reduces their mobility more rapidly.

```

1 score_pair(Center, OppR, OppC,
2             move(MR/MC, RR/RC),
3             Score-move(MR/MC, RR/RC)) :-
4     MDR is abs(MR - Center), MDC is abs(MC - Center),
5     MoveScore is MDR + MDC,      % lower = more central
6     RDR is abs(RR - OppR), RDC is abs(RC - OppC),
7     RemScore is RDR + RDC,      % lower = closer to opponent
8     Score is MoveScore * 3 - RemScore. % weight: move > removal

```

Listing 2.9: Scoring function for move ordering

2.3.4 Evaluation Function

When the search reaches the maximum depth without the game having ended, it is necessary to heuristically evaluate the current position.

Two criteria were selected:

1. **Mobility** — the number of legal moves available. A player with more moves has greater freedom and is further from being isolated. This criterion carries a weight of 10.
2. **Centrality** — how close the pawn is to the centre of the board. The centre offers more escape directions. This criterion serves as a tiebreaker.

```

1 evaluate(Board, P1Pos, P2Pos, Value) :-
2     P1Pos = P1R/P1C, P2Pos = P2R/P2C,
3     valid_moves(Board, P1R, P1C, M1), length(M1, S1), % P1 mobility
4     valid_moves(Board, P2R, P2C, M2), length(M2, S2), % P2 mobility
5     board_size(Board, Size),
6     Center is (Size - 1) / 2.0,
7     centrality(P1R, P1C, Center, Cent1),
8     centrality(P2R, P2C, Center, Cent2),
9     Value is (S1 - S2) * 10 + (Cent1 - Cent2).
10    % positive = good for P1; negative = good for P2

```

Listing 2.10: Heuristic evaluation function

Practical example: on a 3×3 board with P1 at corner (0,0) and P2 at centre (1,1), the calculation is:

- P1 mobility = 2 moves (confined to the corner)
- P2 mobility = 7 moves (centre, maximum freedom)
- Mobility difference: $(2 - 7) \times 10 = -50$
- Centrality: $(0 + 0) - (1 + 1) = -2$ (P2 is closer to the centre)
- Total score: $-50 + (-2) = -52$

The negative value indicates that P2 holds a clear advantage — which is correct: P2 at the centre enjoys considerably greater freedom.

Justification: mobility is the most direct indicator of how close a player is to being isolated. Centrality captures the strategic tendency of maintaining the pawn near the centre of the board.

2.4 Game Interface (game.pl)

2.4.1 Program Flow

The game follows the flow described below:

1. `play/0` — displays the main menu.
2. `main_menu/0` — allows the game mode to be selected (human-human, human-AI, AI-AI).
3. `setup_game/2` — configures the board size and the AI search depth.
4. `start_game/4` — creates the initial board with pawns at opposite corners.

5. `game_loop/6` — main loop: displays the board, checks for game over, and executes the current player's turn.

```

1 game_loop(Board, Turn, P1Pos, P2Pos, Mode, Depth) :-
2     display_game(Board, Turn),
3     ( game_over(Board, Turn, P1Pos, P2Pos)
4     -> Winner is 3 - Turn, % the opponent won
5         announce_winner(Winner, Board),
6         play_again
7     ; execute_turn(Board, Turn, P1Pos, P2Pos,
8                     Mode, Depth,
9                     NewBoard, NewP1, NewP2),
10        NextTurn is 3 - Turn, % alternate: 1->2, 2->1
11        game_loop(NewBoard, NextTurn, NewP1, NewP2,
12                  Mode, Depth)
13    ).

```

Listing 2.11: Main game loop

2.4.2 Human Turn vs. AI Turn

When the turn belongs to a human player, the program requests input in the format `Row/Column`. (with a full stop, since SWI-Prolog interprets the input as a complete Prolog term):

```

Move (row/column, e.g.: 1/2): 1/1.
Remove a square (row/column): 2/2.

```

When the turn belongs to the AI, the predicate `best_move/7` is invoked automatically, executing the alpha-beta search and returning the optimal (move, removal) pair.

2.5 Use of Artificial Intelligence in Development

Throughout the development of this project, generative AI (specifically the Claude model by Anthropic) was used extensively and integrally across all phases of the work. Its use was not limited to code generation, but encompassed the entire learning and development process:

1. **Concept learning:** AI was consulted to understand the fundamentals of Prolog (unification, *backtracking*, lists, built-in predicates) through explanations, examples, and analogies.
2. **Algorithm understanding:** the *minimax* principle and the *alpha-beta* algorithm were explored with AI assistance, which provided step-by-step explanations, tree diagrams, and complexity analysis.
3. **Exploration of alternatives:** different approaches were discussed for board representation (list of lists vs. hash table), for the evaluation function (mobility, centrality, distance between pawns), and for move ordering strategies.

4. **Code writing and debugging:** AI assisted in writing predicates, correcting logical errors (including a bug in the value accumulation within the alpha-beta procedure), and in formulating the 33 unit tests.
5. **Report writing:** AI was used to assist in drafting, structuring, and reviewing this document, with the objective of producing clear and accessible explanations.

In all cases, the use of AI was guided by the pedagogical objectives of the course: AI served as a learning and productivity tool, and not as a substitute for understanding the underlying concepts.

Chapter 3

Experimental Evaluation

This chapter presents the results of the tests performed on the system: unit tests for functional correctness and performance benchmarks under different board size and search depth configurations.

3.1 Unit Tests

The project includes a suite of 33 unit tests (file `tests/tests.pl`), organised into four groups.

3.1.1 Execution and Results

The tests were executed with the command:

```
swipl -g "run_tests, halt" tests/tests.pl
```

The result obtained was:

```
% Start unit: board
% [1/33] board:initial_board_6x6_size ..... passed (0.015 sec)
% [2/33] board:initial_board_6x6_row_length ..... passed (0.001 sec)
% [3/33] board:initial_board_all_free ..... passed (0.000 sec)
% [4/33] board:all_cells_3x3_count ..... passed (0.000 sec)
% [5/33] board:get_cell_after_set ..... passed (0.000 sec)
% [6/33] board:set_cell_nondestructive ..... passed (0.000 sec)
% [7/33] board:get_cell_oob_row ..... passed (0.000 sec)
% [8/33] board:get_cell_oob_col ..... passed (0.000 sec)
% [9/33] board:empty_cell_free ..... passed (0.000 sec)
% [10/33] board:empty_cell_pawn ..... passed (0.000 sec)
% [11/33] board:empty_cell_removed ..... passed (0.000 sec)
% End unit board: passed (0.007 sec CPU)

% Start unit: rules
% [12/33] rules:valid_moves_centre_3x3_count ..... passed (0.000 sec)
```

```
% [13/33] rules:valid_moves_corner_3x3_count ..... passed (0.000 sec)
% [14/33] rules:valid_moves_blocked_by_remove ..... passed (0.000 sec)
% [15/33] rules:valid_moves_no_overlap_opponent ..... passed (0.000 sec)
% [16/33] rules:valid_removes_excludes_pawns ..... passed (0.000 sec)
% [17/33] rules:valid_removes_excl_removed_cell ..... passed (0.000 sec)
% [18/33] rules:apply_move_source_cleared ..... passed (0.000 sec)
% [19/33] rules:apply_remove_marks_cell ..... passed (0.000 sec)
% [20/33] rules:game_over_no_moves ..... passed (0.000 sec)
% [21/33] rules:game_over_has_moves ..... passed (0.000 sec)
% [22/33] rules:player_pos_p1 ..... passed (0.000 sec)
% [23/33] rules:player_pos_p2 ..... passed (0.000 sec)
% End unit rules: passed (0.001 sec CPU)
```

```
% Start unit: ai
% [24/33] ai:evaluate_mobility_advantage ..... passed (0.000 sec)
% [25/33] ai:evaluate_opponent_advantage ..... passed (0.000 sec)
% [26/33] ai:evaluate_symmetric ..... passed (0.000 sec)
% [27/33] ai:terminal_value_p1_loses ..... passed (0.000 sec)
% [28/33] ai:terminal_value_p2_wins ..... passed (0.000 sec)
% [29/33] ai:best_move_trivial_win ..... passed (0.001 sec)
% [30/33] ai:alphabeta_terminal ..... passed (0.000 sec)
% [31/33] ai:minimax_depth_zero ..... passed (0.000 sec)
% End unit ai: passed (0.002 sec CPU)
```

```
% Start unit: integration
% [32/33] integration:scripted_turn_1 ..... passed (0.000 sec)
% [33/33] integration:scripted_game_over_detection .. passed (0.000 sec)
% End unit integration: passed (0.000 sec CPU)
```

```
% All 33 tests passed in 0.029 seconds (0.014 cpu)
```

All 33 tests passed in 0.029 seconds. The fast execution is due to the fact that most tests operate on 3×3 boards or on artificially constructed positions specifically designed for testing.

3.1.2 Test Coverage

3.1.3 Unit Test Example

To illustrate the structure of the tests, the test that verifies the number of valid moves from the centre of a 3×3 board is presented:

```
1 :- use_module(library(plunit)).
2 :- begin_tests(rules).
3
4 test(valid_moves_centre_3x3_count) :-
5     initial_board(3, B0),
```

Table 3.1: Distribution and coverage of unit tests

Module	No. tests	Aspects verified
board	11	Board creation; coordinate-based access; non-destructive update; out-of-bounds access (must fail without error); <code>empty_cell</code> for free, occupied, and removed cells.
rules	12	Valid moves from centre and corner; blocking by a removed square; absence of overlap with the opponent; removal list (excludes pawns and already-removed squares); move and removal application; end-game detection.
ai	8	Evaluation function (P1 advantage, P2 advantage, symmetric position); terminal values; best move in a trivially winning position; alpha-beta at a terminal position; minimax at depth zero.
integration	2	Complete turn (move + removal in a real game); end-game detection in the context of a played game.

```

6   set_cell(B0, 0, 0, x, B1),
7   set_cell(B1, 2, 2, o, Board),
8   valid_moves(Board, 1, 1, Moves), % pawn at centre (1,1)
9   length(Moves, N),
10  assertion(N == 7). % 8 directions - 1 (occupied corner)
11
12 :- end_tests(rules).

```

Listing 3.1: Test: moves from the centre of a 3×3 board

3.2 Performance Benchmarks

To evaluate the performance of the alpha-beta algorithm, the execution times for the first AI move were measured under different combinations of board size and search depth. All tests started from the initial game state (pawns at opposite corners, completely free board).

3.2.1 Methodology

Times were measured using the `get_time/1` predicate from SWI-Prolog (wall-clock time), in the following environment:

- Operating system: macOS Darwin 25.3.0 (Apple Silicon)
- SWI-Prolog (standard version, no additional optimisations)
- Measurement: time to compute the first AI move from the initial state

3.2.2 Results

Table 3.2: Computation time for the first AI move

Board	Depth	Time	Move	Removal
3×3	2	4 ms	(0,0)→(1,1)	(1,2)
3×3	4	136 ms	(0,0)→(1,1)	(1,2)
6×6	2	87 ms	(0,0)→(1,1)	(4,4)
6×6	4	167.619 ms $\approx$ 2 min 47 s	(0,0)→(1,1)	(4,4)

3.2.3 Results Analysis

3.2.3.1 Actual Branching Factor

In the initial state of a 6×6 board, Player 1 can:

- **Move** to 3 squares (corner with 3 free neighbours).
- **Remove** any of 34 free squares (36 squares – 2 pawns).

Total (move, removal) pairs: $3 \times 34 = 102$.

This is the branching factor at level 1. In subsequent moves, as pawns advance towards the centre, each player may have up to 8 possible destinations, increasing the branching factor further.

3.2.3.2 Impact of Depth

The transition from depth 2 to depth 4 on the 6×6 board represents an increase from 87 ms to 167,619 ms — a factor of approximately $1927\times$. This super-exponential growth confirms the combinatorial nature of the problem.

Although alpha-beta significantly reduces the number of explored nodes compared to pure minimax, the high branching factor (≈ 102) in the initial state of a 6×6 board renders depth 4 impractical for interactive use.

3.2.3.3 Consistency of Selected Moves

It is notable that across all configurations, the AI selected the same move: moving the pawn from the corner (0,0) to the centre (1,1). This is consistent with the evaluation function that favours centrality — the pawn at position (1,1) of a 3×3 board can access up to 7 squares, whereas at the corner (0,0) it has only 3.

The difference in the removal choice between depth 2 ((1,2) on 3×3) and depth 4 ((4,4) on 6×6) reflects the deeper lookahead afforded by the greater search depth.

Table 3.3: Evaluation function values at test positions

Position	Configuration	Mob. P1	Mob. P2	Value
Symmetric (opposite corners)	3×3, (0,0) vs. (2,2)	3	3	0
Asymmetric (corner vs. centre)	3×3, (0,0) vs. (1,1)	2	7	−52

3.2.3.4 Heuristic Function Evaluation

The tests confirmed the expected values of the evaluation function:

The value 0 at the symmetric position confirms that the function is unbiased towards either player. The value −52 at the asymmetric position correctly reflects the significant advantage of P2 (greater mobility and better centrality).

3.3 Limitations and Future Work

3.3.1 Identified Limitations

1. **Performance on large boards at high depth:** The 6×6 + depth 4 combination is prohibitive (≈ 167 s per move). The high branching factor, resulting from the large number of removable squares, is the primary cause.
2. **Input format:** human players must enter positions in the format `Row/Column`. (with a full stop), since the `read/1` predicate in SWI-Prolog expects a complete Prolog term terminated by a full stop. This format may cause confusion.
3. **Absence of a transposition table:** identical positions reached via different paths are re-evaluated. A transposition table would reduce redundant computation.
4. **No iterative deepening:** the search depth is fixed. Iterative deepening (gradually increasing the depth up to a time limit) would improve game quality within a fixed time budget.

3.3.2 Configuration Recommendations

Based on the experimental results:

- For 3×3 boards: depth 4 is appropriate (136 ms per move).
- For 6×6 boards: depth 2 is the recommended compromise (87 ms per move), offering a near-instantaneous response.
- Depth 6 is not recommended for boards larger than 3×3 without additional optimisations.

3.4 Conclusions

This work demonstrated that it is possible to implement a complete strategy game with AI in Prolog in an elegant and effective manner.

Objectives achieved:

- Complete implementation of the ISOLA game with all modes (human-human, human-AI, AI-AI) and configurable board sizes.
- Functional alpha-beta algorithm with move ordering and a heuristic evaluation function.
- Suite of 33 unit tests, all passing.
- Practical understanding of *backtracking*, logic programming, and game search algorithms.

Key findings:

- Prolog allows game rules to be expressed almost directly from their natural language description, exploiting *backtracking* to automatically generate all valid moves.
- Alpha-beta is essential to make *minimax* practical: without pruning, depth 4 on a 6×6 board would be entirely infeasible.
- Move ordering is as important as alpha-beta itself: evaluating poor moves first drastically reduces the effectiveness of the cutoffs.

References

- [1] Ivan Bratko, *Prolog Programming for Artificial Intelligence*, Third. Addison-Wesley, 2001.
- [2] Bernd Kienitz, *Isola — isolation strategy board game*, Abstract strategy game for two players, 1972.
- [3] Donald E. Knuth and Ronald W. Moore, “An analysis of alpha-beta pruning”, *Artificial Intelligence*, vol. 6, no. 4, pages 293–326, 1975.
- [4] Stuart Russell and Peter Norvig, *Artificial Intelligence: A Modern Approach*, Fourth. Pearson, 2020.
- [5] Leon Sterling and Ehud Shapiro, *The Art of Prolog: Advanced Programming Techniques*, Second. MIT Press, 1994.

Appendix A

Source Code

This appendix presents the core predicates of each project module. The complete source code is available in the project repository.

A.1 board.pl — Board

```
1 initial_board(Size, Board) :-
2     length(Board, Size),
3     maplist(make_row(Size), Board).
4
5 make_row(Size, Row) :-
6     length(Row, Size),
7     maplist(=('.'), Row).
8
9 get_cell(Board, Row, Col, Cell) :-
10    board_size(Board, Size),
11    Row >= 0, Row < Size, Col >= 0, Col < Size,
12    nth0(Row, Board, RowList),
13    nth0(Col, RowList, Cell).
14
15 set_cell(Board, Row, Col, Value, NewBoard) :-
16    nth0(Row, Board, OldRow, RestRows),
17    nth0(Col, OldRow, _, RestCols),
18    nth0(Col, NewRow, Value, RestCols),
19    nth0(Row, NewBoard, NewRow, RestRows).
20
21 empty_cell('.').
```

Listing A.1: board.pl — board creation and manipulation

A.2 rules.pl — Game Rules

```
1 valid_move(Board, FromR, FromC, ToR, ToC) :-
2     board_size(Board, Size), Max is Size - 1,
3     member(DR, [-1,0,1]), member(DC, [-1,0,1]),
4     \+ (DR == 0, DC == 0),
```

```

5   ToR is FromR + DR, ToC is FromC + DC,
6   ToR >= 0, ToR =< Max, ToC >= 0, ToC =< Max,
7   get_cell(Board, ToR, ToC, '.').
8
9 valid_removes(Board, P1R, P1C, P2R, P2C, Removes) :-
10  board_size(Board, Size), Max is Size - 1,
11  findall(R/C,
12    ( between(0,Max,R), between(0,Max,C),
13      \+(R:=P1R, C:=P1C), \+(R:=P2R, C:=P2C),
14      get_cell(Board, R, C, '.')),
15    Removes).
16
17 apply_move(Board, FromR, FromC, ToR, ToC, NewBoard) :-
18  get_cell(Board, FromR, FromC, Piece),
19  set_cell(Board, FromR, FromC, '.', B1),
20  set_cell(B1, ToR, ToC, Piece, NewBoard).
21
22 apply_remove(Board, Row, Col, NewBoard) :-
23  set_cell(Board, Row, Col, '#', NewBoard).
24
25 game_over(Board, Player, P1Pos, P2Pos) :-
26  player_pos(Player, P1Pos, P2Pos, CR/CC),
27  \+ valid_move(Board, CR, CC, _, _).

```

Listing A.2: rules.pl — move validation and application

A.3 ai.pl — Alpha-Beta Algorithm

```

1 best_move(Board, Player, P1Pos, P2Pos, Depth, MovePos, RemPos) :-
2   alphabeta(Board, Player, P1Pos, P2Pos, Depth,
3     -10000, 10000, _Val, BM, BR),
4   ( BM = none
5     -> any_legal_move(Board, Player, P1Pos, P2Pos,
6       MovePos, RemPos)
7     ; MovePos = BM, RemPos = BR
8     ).
9
10 alphabeta(Board, _Player, P1Pos, P2Pos, 0, _A, _B,
11   Value, none, none) :-
12   !, evaluate(Board, P1Pos, P2Pos, Value).
13
14 alphabeta(Board, Player, P1Pos, P2Pos, _D, _A, _B,
15   Value, none, none) :-
16   player_pos(Player, P1Pos, P2Pos, CR/CC),
17   \+ valid_move(Board, CR, CC, _, _), !,
18   terminal_value(Player, Value).
19
20 alphabeta(Board, Player, P1Pos, P2Pos, Depth, Alpha, Beta,
21   Value, BM, BR) :-
22   all_move_pairs(Board, Player, P1Pos, P2Pos, Pairs),
23   order_pairs(Pairs, Board, Player, P1Pos, P2Pos, Ordered),
24   NextDepth is Depth - 1,
25   Other is 3 - Player,

```

```
26      ( Player := 1
27      ->  ab_max(Ordered, Board, Other, P1Pos, P2Pos,
28                Player, NextDepth, Alpha, Beta,
29                -10000, none, none, Value, BM, BR)
30      ;   ab_min(Ordered, Board, Other, P1Pos, P2Pos,
31                Player, NextDepth, Alpha, Beta,
32                10000, none, none, Value, BM, BR)
33      ).
34
35 evaluate(Board, P1Pos, P2Pos, Value) :-
36     P1Pos = P1R/P1C, P2Pos = P2R/P2C,
37     valid_moves(Board, P1R, P1C, M1), length(M1, S1),
38     valid_moves(Board, P2R, P2C, M2), length(M2, S2),
39     board_size(Board, Size),
40     Center is (Size - 1) / 2.0,
41     centrality(P1R, P1C, Center, Cent1),
42     centrality(P2R, P2C, Center, Cent2),
43     Value is (S1 - S2) * 10 + (Cent1 - Cent2).
```

Listing A.3: ai.pl — entry point and alpha-beta search

